

Security in Software Applications

Java Programming Rules



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Java security guarantees

- The Java platform offers security features
 - memory safety
 - strong typing
 - visibility restrictions (public, private,...)
 - sandboxing based on stackwalking
- This allows security guarantees to be made
 - even if part of the code is untrusted – or simply buggy.
- Of course, the same goes for Microsoft .NET/C#

Programming guidelines

Even if

- type system is sound
- VM & type checker are correct
- sandboxing is sound & implemented correctly
 - ie no exploitable bugs in platform API classes like `java.lang.Class`, `java.lang.SecurityManager`
- policy file is correct

a particular Java class may still be vulnerable to attacks by untrusted code

Programming guidelines have been proposed to prevent known vulnerabilities

Spot the defect

```
public class String {  
    public char[] contents; public int offset, len;  
    // idea: String is contents[offset ... offset+len]  
    String() {contents=null; offset=0; len=0;}  
    String(char[] a) {  
        contents = a; offset = 0; len = a.len; }  
    String substring(int take) {  
        if (take<=0) throw new NegativeSizeException();  
        String s = new String();  
        s.content=this.content;  
        s.offset=0; s.len=Math.min(take,s.len);  
        return s;  
    }  
    int getLength() { return len; }
```

class should be final to prevent malicious subclasses

fields should be not be public to prevent unauthorised changes, ie. preserve integrity

```
public class String {  
    public char[] contents; public int offset, len;
```

```
    // idea: String is contents[offset ... offset+len]
```

```
    String() {contents=null; offset=0; len=0;}  
    String(char[] a) {
```

```
        contents = a; offset = 0; len = a.len; }
```

```
    String substring(int take) {
```

```
        if (take<=0) throw new NegativeSizeException();
```

```
        String s = new String();
```

```
        s.content=this.content;
```

```
        s.offset=0; s.len=Math.min(take,s.len);
```

```
        return s;
```

```
    }
```

```
    int getLength() { return len; }
```

fields should be final (for thread-safety, see next slide)

array should be cloned to prevent representation exposure which allows unauthorised changes

Java security programming guidelines

Sources

- Twelve rules by McGraw & Felten
- Java secure programming HOWTO by Wheeler
-

NB the context of these rules: only really relevant for

- applications that are – or may some day be – extended with less trusted or untrusted code
- API components that are by definition extended with less trusted code

Java security programming guidelines

- Do not use public fields

Java security programming guidelines

- Do not use *protected* fields
- *Why?* Protected fields can be accessed in
 1. subclasses
 2. in the package

Anyone can (1) create subclasses or (2) extend the package
- We can rule out 1 by making a class final
- We can rule out 2 by making a package *sealed*
 - introduced in Java 1.4 to prevent package being extended; java.lang was already sealed by default...

Java security programming guidelines

Hence:

- Do not rely on package protection for security
- in particular: make all fields private
- NB this conflicts another standard Java coding guideline, namely
 - do not make fields private, make them protected instead

This guideline is motivated by desire for extensibility, not security

Java security programming guidelines

- Limit access to classes, methods, and fields
 - make them **private**, **unless...** there is a need for them to be more visible
- cf principle of least privilege
- A compiler only complains if visibility is too restrictive, not if it is too liberal. The JAMIT tool checks if visibility can be made more restrictive.

Potential visibility loophole: reflection

Reflection

= ability of a program to inspect & modify itself

```
Class cl = o.getClass(); // o's class
Field[] fs = cl.getDeclaredFields(); //
for (Field f : fs){
    if (!(Modifier.isPublic()(f.getModifiers()))
        { f.setAccessible(true); }
    print("Field " + f.getName()); println(" has
    value " + f.get(o));
}
```

There is also a method to set fields, `f.set(o, val)`

Potential visibility loophole: reflection

- Reflection allows access to fields that are normally *not* visible
- The default security manager prevents this, and will throw an **AccessControlException**
 - unless `java.lang.reflect.ReflectPermission` is granted

Java security programming guidelines

- Make classes, methods, fields **final**, unless... there is a need for them to be subclassed/overridden/modified
- *Why?*
- cf principle of least privilege

Java security programming guidelines

- Never return (a reference to) a mutable object (incl. arrays) to untrusted code
- Never store a reference to a mutable object (incl. arrays) obtained from untrusted code
- *Why?*

Java security programming guidelines

- Don't use inner classes.
- *Why? Byte code – and byte code verifier - don't know about inner classes; in byte code:*
 - *inner class becomes package visible normal class*
 - *private field of outer class may become package visible*

Example inner class

```
public class Outer{ private int x = 5;
    private class Inner {
        int helper(){ return x; }
    }
    public int getX(){
        Inner i = new Inner();
        return i.helper();
    }

    public static void main(String[] args) { Outer o
        = new Outer(); System.out.println(o.getX());
    }
}
```


Example inner class

- Make classes non-cloneable

Java security programming guidelines

- Make classes non-deserialiseable.
- Make classes non-serialiseable.
- I.e. don't implement

```
void writeObject(java.io.ObjectOutputStream)
void readObject(java.io.ObjectOutputStream)
```
- *Why?*

Java security programming guidelines

General observations

- Flexibility & extensibility provided by OO is nice for programming, but nasty for security
 - usual tension between functionality & security
- These issues are not at all obvious, and typical programmers won't know these things, unless they have actively looked for information
 - see principle: use your community resources,
eg google for language/setting specific security guidelines

More programming guidelines

- There are more programming guidelines, that are not directly motivated by security.
- Specific APIs may come with their own programming guidelines, possibly security-related
- Guidelines are very useful as checklists, but do not be tempted into thinking they are complete and give absolute guarantees.

More language-based security

Ongoing work for

- Richer type systems, eg
 - alias protection to prevent MagicCoat,
 - more on alias protection later
 - non_null references
 - Immutable objects
- Tainting / information flow
 - eg taint mode in perl